GoServer

Project team

October 23, 2026

Contents

1 Project sketch **1**

1.1 People and systems . . . 1
1.2 Primary navigation . . . 1
1.3 Initial game format . . . 2
1.4 Teams and alliances . . . 2
1.5 Bots . . . 3
1.6 Chat . . . 3
1.7 Representative scenarios . . . 3
1.7.1 Find and enter a game . . . 3
1.7.2 Play a timed move . . . 3
1.7.3 Capture neighboring groups . . . 3
1.7.4 Play a suicide move . . . 3
1.7.5 Fill a seat with a bot . . . 4
1.7.6 Form an alliance . . . 4
1.7.7 Score a team . . . 4
1.7.8 Chat inside a game . . . 4

2 Requirements argument **5**

2.1 Observable scope . . . 5
2.2 Proposed safety and consistency invariants . . . 5
2.3 Resolved capture and suicide ordering . . . 5
2.4 Resolved scoring direction . . . 6
2.5 Resolved bot execution direction . . . 6
2.6 Recovered ko and repetition behavior . . . 6
2.7 Open rules decisions . . . 7

3 Implementation argument **8**

Chapter 1

Project sketch

GoServer is a KGS-inspired online Go service focused on multiplayer variants rather than direct competition with established two-player Go servers. The initial release centers on short, four-participant games played on an 11×11 board.

KGS provides inspiration for the navigation model, but this sketch does not require compatibility with KGS accounts, protocols, clients, or server internals.

1.1 People and systems

The primary participant is a signed-in person who wants to browse chat rooms, find or observe games, play Go, and converse with other participants. A game seat may instead be controlled by a server-supported bot. People and bots are both game participants for the purposes of turn order and team membership.

The server maintains identities, sessions, rooms, games, board state, clocks, team membership, participant graveyards, and chat history. Connected clients present server-authoritative state and submit participant actions.

1.2 Primary navigation

After a successful login, a person sees the available chat rooms. Entering a room displays the room conversation and the games associated with that room. Entering a game displays its board, participants, teams, move deadline, current turn, game status, and game-specific chat.

The intended hierarchy is:

1. authenticate;
2. browse and enter a chat room;
3. inspect the games associated with that room;
4. enter a game as a player or observer; and
5. play or observe while using the game chat.

Room chat and game chat are separate conversations. A message sent to one is not implicitly copied to the other.

1.3 Initial game format

The first supported format has the following fixed profile:

- an 11×11 Go board;
- four participant seats;
- sequential turns;
- a ten-second deadline for each move;
- each seat configurable as either a person or a bot; and
- each participant initially occupying a distinct team.

The server is authoritative for legal moves, turn order, deadlines, captures, repetition rules, team membership, graveyard contents, and the final result. Clients must not resolve conflicting game actions locally.

When a stone is placed, the server first examines its orthogonally adjacent groups. Every adjacent group that has no remaining liberties is removed before the server evaluates the placed stone for suicide. This ordering permits an otherwise surrounded placement to survive when it first captures neighboring stones and thereby gains liberties.

If no neighboring group is removed and the placed stone's connected same-owner group has no liberties, the move is a permitted suicide move. The server removes the newly placed stone and its entire connected same-owner group. This permits multiplayer tactics in which a participant removes their own group to help an allied participant form eyes. Every self-removed stone is placed in its owner's graveyard, just like a stone removed by another participant.

1.4 Teams and alliances

Every participant begins the game as the sole member of a separate team. Participants may subsequently join together so that multiple participants belong to one team. A team therefore represents an alliance of one or more seats rather than a single shared seat.

A team merge changes cooperation and scoring without removing a participant's seat or turn. Each participant continues to place stones during that participant's assigned turns. Existing stones remain on the board when teams merge.

For each participant, the server maintains a graveyard containing that participant's stones that have been removed from play. When, for example, White captures a Black stone, the removed stone is given to the Black participant's graveyard rather than credited to White. A stone removed by its owner's suicide is likewise placed in that owner's graveyard. Removal therefore does not directly award points to another participant.

A team's score is its total territory minus its total losses. Total losses are the number of removed stones in the graveyards of the team's members. Equivalently, if A_T is the territory attributed to team T and G_T is the combined number of stones in its members' graveyards, then

$$S_T = A_T - G_T.$$

This makes the result express territory gained at the cost of stones lost. The approval, timing, and reversibility of a team merge remain requirements decisions.

1.5 Bots

Each of the four seats can be assigned independently to a person or bot. This permits any mixture from four people to four bots. Computer-controlled seats use Monte Carlo tree search executed by the server. Clients do not run the authoritative bot search or submit bot-selected moves on behalf of a bot seat.

A bot is subject to the same legal-move checks and ten-second move deadline as a person. Its server-side search must stop when its turn expires, and a result that arrives after the authoritative deadline must not advance the game. Bot failures must not give the bot permission to bypass the clock, acquire privileges unavailable to its assigned seat, or submit moves after its turn has ended. The implementation must bound and isolate bot computation so that one search cannot exhaust resources needed by unrelated games.

1.6 Chat

A participant can converse in a room without entering a game. Every game also has a dedicated chat visible to people who have entered that game, subject to the eventual observer and moderation policy. Bot participation in chat is not required by the initial sketch.

1.7 Representative scenarios

1.7.1 Find and enter a game

A person logs in, sees the room list, enters a room, and receives the current room conversation and game list. The person selects a game and receives its board, participants, teams, clock state, graveyard counts, and game chat.

1.7.2 Play a timed move

When a participant's turn begins, the server starts a ten-second deadline. The participant submits a board coordinate. The server accepts the move only if the participant controls the active seat, the turn is still current, the deadline has not expired, and the move is legal. The accepted state is then distributed to everyone viewing the game.

1.7.3 Capture neighboring groups

A participant places a stone next to one or more groups that have lost their final liberty. The server removes all such neighboring groups before evaluating whether the newly placed stone's group has liberties. Each removed stone is placed in the graveyard of the participant who owned it. No capture point is awarded to the participant who placed the stone.

1.7.4 Play a suicide move

A participant places a stone that does not remove any neighboring group, and the placed stone's connected same-owner group is left without liberties. The server treats the move as a permitted suicide and removes the placed stone together with its entire connected same-owner group. Every self-removed stone is placed in its owner's graveyard and therefore contributes to that owner's losses.

1.7.5 Fill a seat with a bot

A game is configured with one or more bot-controlled seats. When a bot-controlled turn begins, the server runs that bot's Monte Carlo tree search within server-controlled execution. The search operates under the same ten-second deadline and move-legality constraints as a human-controlled seat. Only a result selected before the authoritative deadline may be submitted for the current turn.

1.7.6 Form an alliance

Participants who began on separate teams request to become members of the same team. The server applies the selected merge policy, records the resulting membership, and distributes the updated team state to all viewers. Existing stones remain on the board. The resulting team's loss total includes the graveyards of all its members.

1.7.7 Score a team

At the end of a game, the server totals the territory attributed to a team and the removed stones held in its members' graveyards. It subtracts the graveyard total from the territory total and reports the resulting team score. Captures and suicides affect scoring only as losses borne by the owners of the removed stones.

1.7.8 Chat inside a game

A person viewing a game sends a message to the game chat. The server associates the message with that game and distributes it to authorized game viewers without adding it to the containing room's chat.

Chapter 2

Requirements argument

2.1 Observable scope

The candidate first-release requirements will cover authentication, room discovery, room entry, room chat, room-associated game discovery, game entry, game chat, four-seat game setup, human and bot seats, server-executed bot search, server-authoritative timed turns, multiplayer Go rules, participant graveyards, team merges, scoring, reconnect behavior, and observable error responses.

Account registration, moderation administration, persistent ratings, tournaments, KGS protocol compatibility, and sophisticated bot strategy beyond the required server-side Monte Carlo tree search are not yet part of the committed first-release scope.

2.2 Proposed safety and consistency invariants

The requirements should preserve the following positive invariant: every externally visible room, chat, game, move, clock, capture, graveyard, score, and team transition is authorized for the acting identity and derived from one unambiguous server-authoritative state.

Candidate forbidden behaviors include accepting a move from the wrong seat, accepting more than one move for a turn, accepting a move after its deadline, resolving suicide before neighboring captures, removing less or more than the entire connected same-owner group during suicide, crediting a removed stone to another participant, placing a removed stone in the wrong owner's graveyard, allowing a client to manufacture team membership or graveyard contents, accepting a client-supplied move as the output of a server-controlled bot, leaking private session credentials into chat or logs, crossing room or game boundaries through an identifier substitution, letting a bot acquire privileges unavailable to its assigned seat, and allowing one bot search to consume unbounded server resources.

These statements are design candidates rather than accepted requirements until represented by the typed project specification and explicitly accepted.

2.3 Resolved capture and suicide ordering

After a stone is placed, adjacent groups without liberties are removed first. Only if no neighboring group is removed does the server evaluate the placement as suicide. Thus a placement that captures a neighboring group can gain the liberties created by that removal and remain on the board.

Suicide is permitted as a multiplayer tactic, including to help an ally form eyes. If the placed stone's connected same-owner group has no liberties, the server removes the placed stone and that

entire connected group. All self-removed stones enter their respective owners' graveyards and count as losses in exactly the same way as stones removed by another participant.

2.4 Resolved scoring direction

A removed stone is assigned to the graveyard of the participant who owned it. It counts as one loss for that participant and is not a positive capture point for another participant.

A team's final score is its total territory minus the combined number of stones in its members' graveyards. Graveyard ownership remains attached to the participant, allowing team losses to be recomputed from authoritative participant records after a team merge. The typed specification must still define how territory is identified and attributed.

2.5 Resolved bot execution direction

Computer-player Monte Carlo tree search runs on the server rather than in a connected client. The authoritative game clock bounds the search, and a late search result cannot advance an expired or superseded turn. The implementation mapping must define resource bounds and execution isolation for simultaneous searches without changing the observable ten-second deadline.

2.6 Recovered ko and repetition behavior

The supplied N-Player Go Lab JavaScript provides concrete provenance for the intended ko behavior. Its user-facing description calls the rule "simple ko plus positional superko," but its state comparison includes both the board and the participant to move. The standard description of the implemented rule is therefore *situational superko*, not positional superko.

The candidate first-release baseline is as follows:

- the ko rule is enabled by default for a two-participant game and disabled by default for a game with three or more participants;
- when enabled, a qualifying single-stone capture establishes a simple-ko restriction that prevents the owner of the captured stone from immediately recapturing at that point on that participant's next turn;
- after every proposed stone placement, the resulting board together with the next participant to move must differ from every earlier such state on the current game line;
- passes, deadline expirations, and resignations are not rejected merely because they leave or recreate a repeated state; and
- independently of repetition, a game is capped at $3N^2$ moves for an $N \times N$ board.

The prototype represents a repetition state with a 32-bit Zobrist board hash combined with a token for the next participant. That representation is useful evidence of semantics but is not sufficient as the sole authoritative equality test because a hash collision could reject a legal position as repeated. The production requirement should demand collision-safe comparison, such as an exact canonical state representation or a digest backed by exact equality when hashes match.

The repetition check applies to actual game-tree branches and is mirrored inside Monte Carlo tree search: a searched stone placement is skipped when its resulting board-and-next-participant

state occurs in the real game history or in the current search path. This search behavior is advisory; the authoritative game transition must independently enforce the same observable rule.

2.7 Open rules decisions

The remaining core multiplayer rules still need durable answers for:

- whether the ko rule is fixed for the first-release four-participant format or configurable when a game is created;
- whether an expired deadline causes an automatic pass, forfeits the seat, or has another result;
- how territory is identified and attributed among participants and teams;
- when team merges are allowed, who must approve them, and whether they are reversible;
- whether observers may use game chat; and
- what reconnect grace, if any, exists within a ten-second turn.

Chapter 3

Implementation argument

Implementation is blocked until the observable behavior is encoded as a typed LeanFM requirement specification, checked, and explicitly accepted, followed by a checked and explicitly accepted implementation mapping.

Once those gates are satisfied, the implementation argument must map authentication, authorization, room isolation, game isolation, clocks, turn serialization, legal-move validation, ordered capture and suicide processing, graveyard attribution, collision-safe situational-superko validation, team transitions, server-side Monte Carlo tree search, bot resource isolation, persistence, chat delivery, and scoring attribution to concrete controls and tests. In particular, concurrent move and timeout processing must have a single atomic winner so that a deadline and a move cannot both advance the same turn. A bot result must additionally be bound to its game, seat, turn revision, and deadline so that a delayed or confused result cannot be applied to another turn. A placement, all neighboring removals, any resulting connected-group suicide removal, repetition validation, and all corresponding board, graveyard, and score-relevant changes must be committed as one authoritative transition.